

Relatório Técnico / Análise Final — Guryflix

Feito Por: Martim Peixe

Turma: TGPSI24-T

Ultima Data alterada: 27/06/2026



Este relatório descreve o planeamento, a arquitetura e os detalhes de implementação do projeto **Guryflix**, uma plataforma de streaming de trailers de filmes e gestão de catálogo de dados.

1. Contextualização e Objetivos

O projeto **Guryfli*** foi concebido como uma aplicação desktop interativa inspirada na interface da plataforma Netflix. O principal objetivo é permitir a utilizadores finais a criação de contas, gestão de perfis personalizados, definição de preferências, catalogação de filmes e reprodução de trailers online (streaming).

Durante a fase de desenvolvimento, o sistema foi evoluído de um armazenamento inicial em ficheiros de texto para uma base de dados relacional robusta (SQL Server), com a adição de uma interface administrativa para manutenção de todos os dados do catálogo (CRUD).

2. Arquitetura de Software e Tecnologias

A aplicação segue uma arquitetura cliente-servidor de duas camadas (Client-Server 2-Tier):

- **Camada de Apresentação (UI):** Desenvolvida em C# Windows Forms (.NET Framework 4.8) com design moderno em modo escuro, efeitos de animação no cursor e controlos dinâmicos.
- **Camada de Dados (BD):** SQL Server local (suporte automático a LocalDB e SQL Express) acedido via ADO.NET (`System.Data.SqlClient`).
- **Segurança:** Criptografia de palavras-passe utilizando a biblioteca BCrypt (algoritmo adaptativo KDF de hash unidirecional).

3. Modelo de Dados e Normalização

O modelo relacional do Guryflix é composto por **6 tabelas**, estruturadas de forma a respeitar as normas de normalização (até à Terceira Forma Normal - 3NF) para eliminar redundâncias e inconsistências.

FILMES		
int	id	PK
string	titulo	
string	genero	
int	ano	
string	afinidade	
string	sinopse	
string	url_video	

Tabelas do Sistema:

1. **`contas`** : Armazena as credenciais de autenticação principal.
 - Campos: ``id`` (PK, Identity), ``nome_utilizador`` (Unique), ``senha_hash``.
2. **`perfis`** : Armazena perfis individuais criados para cada conta (até 5 por conta).
 - Campos: ``id`` (PK, Identity), ``conta_id`` (FK para contas), ``nome_perfil``, ``senha_hash`` (PIN opcional).
3. **`preferencias`** : Guarda os géneros preferidos selecionados para cada perfil.
 - Campos: ``id`` (PK), ``perfil_id`` (FK para perfis), ``genero``.
4. **`historico`** : Regista os filmes visualizados por perfil e a data.
 - Campos: ``id`` (PK), ``perfil_id`` (FK para perfis), ``titulo_filme``, ``data_visualizacao``.
5. **`videos\_curtidos`** : Regista os filmes marcados como favoritos por cada perfil.
 - Campos: ``id`` (PK), ``perfil_id`` (FK para perfis), ``titulo_filme``, ``data_curtida``.
6. **`filmes`** : Tabela de catálogo contendo as informações de metadados dos filmes.
 - Campos: ``id`` (PK), ``titulo`` (Unique), ``genero``, ``ano``, ``afinidade``, ``sinopse``, ``url_video``.

Justificação da Integridade Referencial:

Todas as relações possuem chaves estrangeiras configuradas com restrições ``ON DELETE CASCADE``. Desta forma, se uma conta de utilizador for eliminada, todos os seus perfis, preferências, histórico e curtidas associadas são automaticamente removidos no SQL Server, garantindo a integridade dos dados e evitando registos órfãos.

4. Detalhes de Implementação Técnica

Ligação Dinâmica ao SQL Server

Para garantir compatibilidade em ambientes escolares e domésticos, a classe `DatabaseContext` testa sequencialmente as strings de conexão para LocalDB e SQL Express. A base de dados e as tabelas são criadas em tempo de execução na primeira inicialização da aplicação:



```
string[] masterConnStrings = new string[] {  
    @"Server=(localdb)\MSSQLLocalDB;Database=master;Integrated Security=True;",  
    @"Server=.\SQLEXPRESS;Database=master;Integrated Security=True;"  
};
```

Interface de Administração CRUD

A tela `AdminPanel` permite gerir e visualizar em tempo real o estado de todas as tabelas através de controlos `DataGridView`. O administrador pode adicionar novos filmes com links de trailer do YouTube, editar as sinopses de filmes existentes ou eliminar filmes obsoletos. O sistema propaga as operações de leitura e gravação de forma segura e parametrizada para prevenir ataques de SQL Injection.

5. Avaliação do Cumprimento de Requisitos

A tabela seguinte avalia os requisitos propostos para o projeto:

ID	Descrição do Requisito	Estado	Observações / Evidência
REQ001	Interface em Windows Forms com C#	Cumprido	Toda a UI desenvolvida no .NET 4.8 WinForms.
REQ002	Base de dados SQL Server integrada	Cumprido	Base de dados criada e gerida no SQL Server local.
REQ003	Modelo de dados com pelo menos 3 tabelas (excluindo users/perfis)	Cumprido	Inclui as tabelas `filmes`, `historico`, `videos_curtidos` e `preferencias`.
REQ004	Ecrã de Login e Autenticação de utilizador	Cumprido	Form `LoginForm` com encriptação BCrypt.
REQ005	Operações CRUD completas (Create, Read, Update, Delete)	Cumprido	Painel administrativo `AdminPanel` com CRUD completo de filmes.
REQ006	Manutenção de Utilizadores/Perfis	Cumprido	Criação e eliminação de contas no `AccountInfo` e perfis no `ProfilesHandling`.
REQ007	Visualização de trailers (Vídeo Player)	Cumprido	Abertura dos links do YouTube armazenados na base de dados no browser padrão.